



SOLID Principles

SOFTWARE ENGINEERING II – LAB 2

What is a Bad Design?

- ▶ There are 3 important characteristics that you should avoid in your software design
 - ▶ **Rigidity** => Hard to change design, every change affects too many parts in the design
 - ▶ **Fragility** => When you make a change in the design, unexpected parts of the system broke
 - ▶ **Immobility** => The design components are hard to **reuse** in another application these components are coupled with each other and cannot be disentangled from the system easily

SOLID Principles

- ▶ **S.O.L.I.D** is an acronym for the **first five object-oriented design(OOD)** principles**** by **Robert C. Martin**
- ▶ These principles gives the following advantages:
 - ▶ Design becomes easy to be extended and maintained.
 - ▶ Avoid code smells (Weakness points in code)
 - ▶ Code Reusability
 - ▶ Easily refactor code

SOLID Principles

- ▶ **S => Single responsibility Principle:** A class should have only one reason to change
- ▶ **O => Open close Principle:** Software entities like classes, modules and functions should be *open for extension but closed for modifications*.
- ▶ **L => Liskov substitution principle:** Derived types must be completely substitutable for their base types
- ▶ **I => Interface segregation principle:** Clients should not be forced to depend upon interfaces that they don't use
- ▶ **D => Dependency inversion principle:** Abstractions should not depend on details. Details should depend on abstractions

S - Single Responsibility principle

- ▶ **Each class should have one responsibility, one single purpose**
- ▶ This means that a class will do only one job, which leads us to conclude it should have **only one reason to change**

Example

```
class Order
{
    public function calculateTotalSum() { }
    public function.getItems() { }
    public function getItemCount() { }
    public function addItem($item) { }
    public function deleteItem($item) { }

    public function printOrder() { }
    public function showOrder() { }

    public function load() { }
    public function save() { }
    public function update() { }
    public function delete() { }
}
```

Solution

```
class Order
{
    public function calculateTotalSum() { }
    public function.getItems() { }
    public function getItemCount() { }
    public function addItem($item) { }
    public function deleteItem($item) { }
}

class OrderRepository
{
    public function load($orderId) { }
    public function save($order) { }
    public function update($order) { }
    public function delete($order) { }
}

class OrderViewer
{
    public function printOrder($order) { }
    public function showOrder($order) { }
}
```

Example

```
01  public class Task
02  {
03      public void downloadFile(location)
04      {
05          //Download the file
06      }
07      public void parseTheFile(file)
08      {
09          //Parse the contents of the file- XML/JSON/CSV
10      }
11      public void persistTheData(data)
12      {
13          //Persist the data to Database or file system.
14      }
15  }
```


Solution

- ▶ Create a separate class for handling each functionality
 - ▶ Class for downloading files
 - ▶ Class for parsing files
 - ▶ Class for database operations

O – Open Close Principle

- ▶ **Objects or entities should be open for extension, but closed for modification.**
- ▶ A class is closed, since it may be compiled, stored in a library, and used by client classes.
- ▶ But it is also open, since any new class may use it as parent, adding new features.
- ▶ When a descendant class is defined, there is no need to change the original or to disturb its clients.”

Example

```
// Open-Close Principle - Bad example
class GraphicEditor {
    public void drawShape(Shape s) {
        if (s.m_type==1)
            drawRectangle(s);
        else if (s.m_type==2)
            drawCircle(s);
    }
    public void drawCircle(Circle r) {....}
    public void drawRectangle(Rectangle r) {....}
}

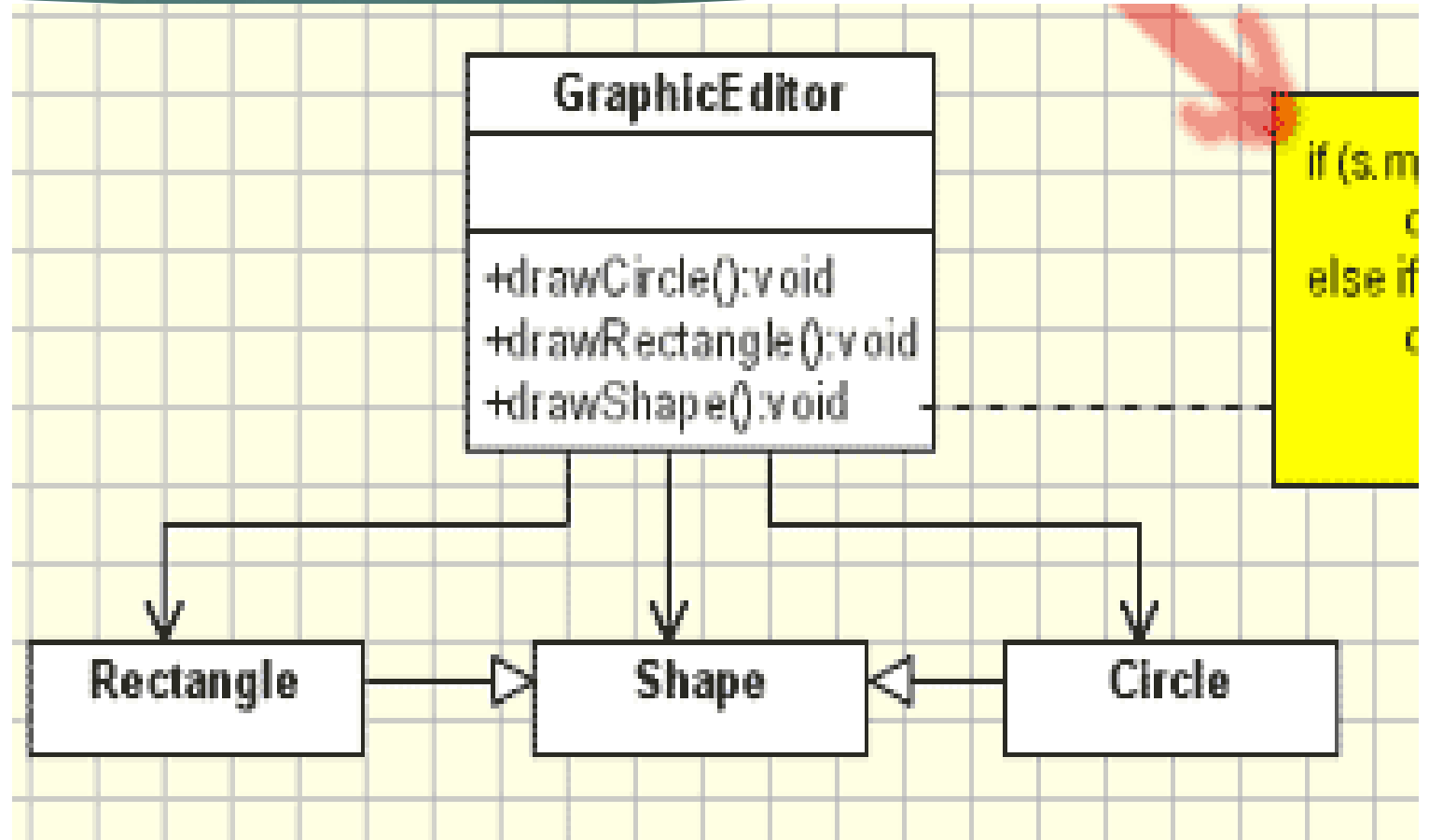
class Shape {
    int m_type;
}

class Rectangle extends Shape {
    Rectangle() {
        super.m_type=1;
    }
}

class Circle extends Shape {
    Circle() {
        super.m_type=2;
    }
}
```

Problem

- This breaks the Open Close Principle.

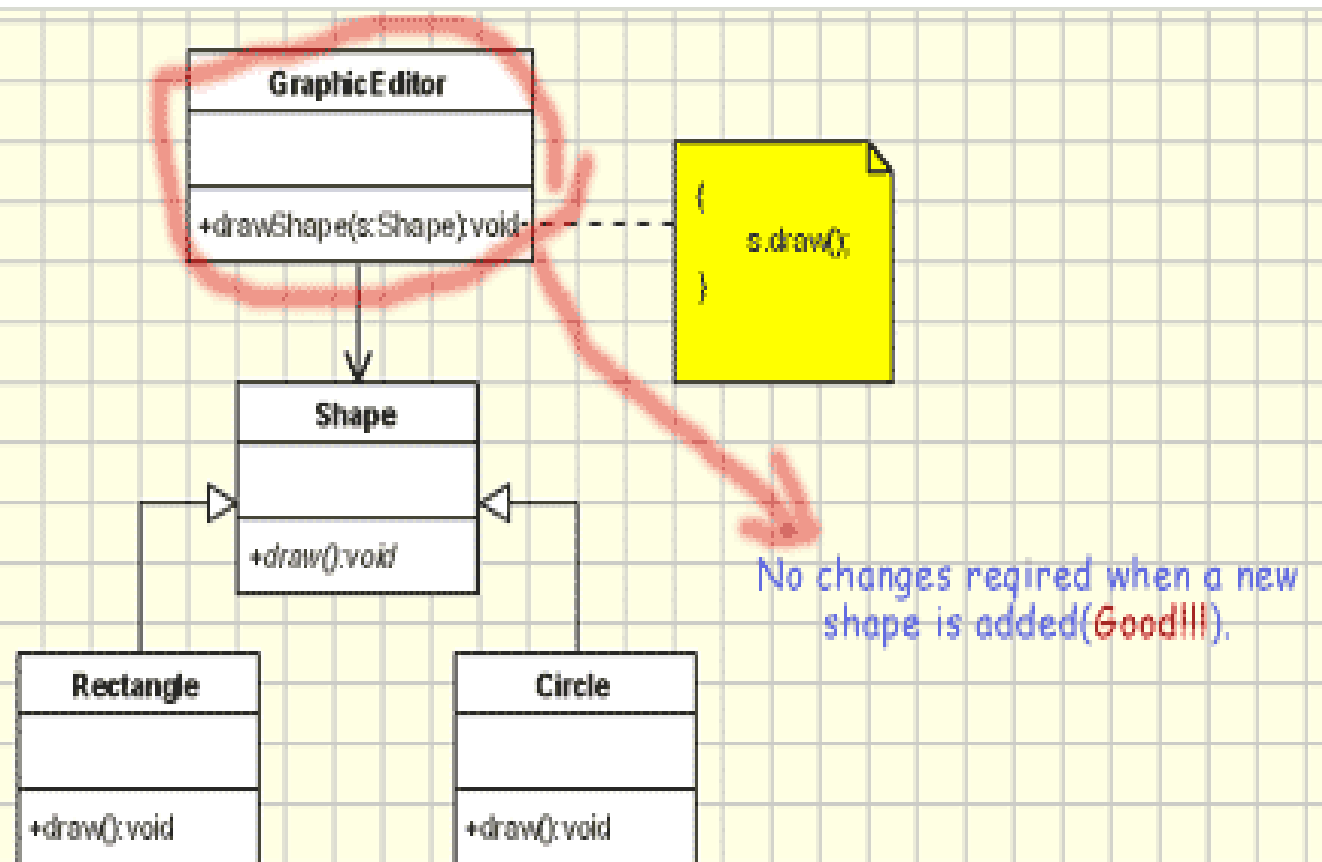


Solution

```
// Open-Close Principle - Good example
class GraphicEditor {
    public void drawShape(Shape s) {
        s.draw();
    }
}

class Shape {
    abstract void draw();
}

class Rectangle extends Shape {
    public void draw() {
        // draw the rectangle
    }
}
```



Example

Calculator.java

```
1
2 public class Calculator {
3
4     private int left;
5     private int right;
6
7     // Setters & Getters
8
9
10    public int Calculate (String operator) {
11
12        int res = 0;
13
14        if (operator.equals("+"))
15            res = left + right;
16
17        else if (operator.equals("-"))
18            res = left - right;
19
20        else;
21        |
22        return res;
23    }
24
25 }
26
```

Solution

```
1 public interface CalculatorOperation {  
2     void perform();  
3 }
```

As a result, the *Addition* class could implement the logic of adding two numbers:

```
1 public class Addition implements CalculatorOperation {  
2     private double left;  
3     private double right;  
4     private double result;  
5  
6     // constructor, getters and setters  
7  
8     @Override  
9     public void perform() {  
10         result = left + right;  
11     }  
12 }
```

Solution

```
1 public class Division implements CalculatorOperation {
2     private double left;
3     private double right;
4     private double result;
5
6     // constructor, getters and setters
7     @Override
8     public void perform() {
9         if (right != 0) {
10             result = left / right;
11         }
12     }
13 }
```

And finally, our *Calculator* class doesn't need to implement new logic as we introduce new operators:

```
1 public class Calculator {
2
3     public void calculate(CalculatorOperation operation) {
4         if (operation == null) {
5             throw new InvalidParameterException("Cannot perform operation");
6         }
7         operation.perform();
8     }
9 }
```


L – Liskov Substitution Principle

- ▶ Derived types must be completely substitutable for their base types
- ▶ if class *A* is a subtype of class *B*, then we should be able to replace *B* with *A* without disrupting the behavior of our program.

// Violation of Likov's Substitution Principle

class Rectangle

```
{
    protected int m_width;
    protected int m_height;

    public void setWidth(int width){
        m_width = width;
    }

    public void setHeight(int height){
        m_height = height;
    }

    public int getWidth(){
        return m_width;
    }

    public int getHeight(){
        return m_height;
    }

    public int getArea(){
        return m_width * m_height;
    }
}
```

class Square extends Rectangle

```
{
    public void setWidth(int width){
        m_width = width;
        m_height = width;
    }

    public void setHeight(int height){
        m_width = height;
        m_height = height;
    }
}
```

Solution

- ▶ **Remove Inheritance between square and rectangle and generalize them by creating abstract class shape.**

I – Interface Segregation Principle

- ▶ A client should never be forced to implement an interface that it doesn't use or clients shouldn't be forced to depend on methods they do not use.

Example

- ▶ Suppose we have Square, Rectangle and Cuboid. All these implements the shape interface.
- ▶ Suppose the Shape interface has calculateArea and calculateVolume methods.

Problem

- ▶ But Square and Rectangle doesn't have volumes. So, make a Solid interface that has volume and just Cuboid implements it.

Solution

```
interface ShapeInterface {  
    public function calculateArea();  
}  
  
interface SolidShapeInterface {  
    public function calculateVolume();  
}
```

```
class Square implements ShapeInterface{  
    public function calculateArea () {  
        // calculate the surface area of the cuboid  
    }  
}  
  
class Cuboid implements ShapeInterface, SolidShapeInterface {  
    public function calculateArea () {  
        // calculate the surface area of the cuboid  
    }  
  
    public function calculateVolume() {  
        // calculate the volume of the cuboid  
    }  
}
```

D – Dependency Inversion Principle

- ▶ High-level modules should not depend on low-level modules. Both should depend on abstractions.
- ▶ Abstractions should not depend on details. Details should depend on abstractions
- ▶ we can consider the low level classes the classes which implement basic and primary operations(disk access, network protocols,...)
- ▶ High level classes the classes which encapsulate complex logic(business flows, ...)

Example

```
class PasswordReminder {  
    private $dbConnection;  
  
    public function __construct(MySqlConnection $dbConnection) {  
        $this->dbConnection = $dbConnection;  
    }  
}
```

Problem

- ▶ What if Oracle DB is Used?? All parts that use MySqlConnection will be affected.
- ▶ Develop a general interface for any DB Connection\
- ▶ Then develop as much classes as the types of the possible DB Connections. Each class should implements its own connect function

Solution

```
interface DBConnectionInterface {  
    public DBConnectionInterface connect();  
}  
class MySQLConnection implements DBConnectionInterface {  
    public DBConnectionInterface connect() {  
        return new MySQLConnection();  
    }  
}  
  
class OracleConnection implements DBConnectionInterface {  
    public DBConnectionInterface connect() {  
        return new OracleConnection();  
    }  
}  
  
class PasswordReminder {  
    private DBConnectionInterface dbConnection;  
  
    public construct(DBConnectionInterface dbConnection) {  
        this.dbConnection = dbConnection;  
    }  
}
```

Let's put it all together

- ▶ We are writing a Java application for a **book store**. We create a Book class that lets users get and set the titles and authors of each book, and search the book in the inventory.

The Book Class

```
class Book {  
  
    String title;  
    String author;  
  
    String getTitle() {  
        return title;  
    }  
    void setTitle(String title) {  
        this.title = title;  
    }  
    String getAuthor() {  
        return author;  
    }  
    void setAuthor(String author) {  
        this.author = author;  
    }  
    void searchBook() {...}
```

However, the above code violates **the Single Responsibility Principle**, as the Book class has two responsibilities. First, it sets the data related to the books (title and author). Second, it searches for the book in the inventory. The setter methods change the Book object, which might cause problems when we want to search the same book in the inventory.

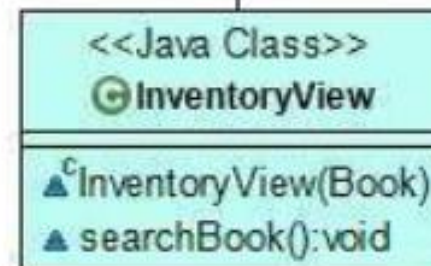
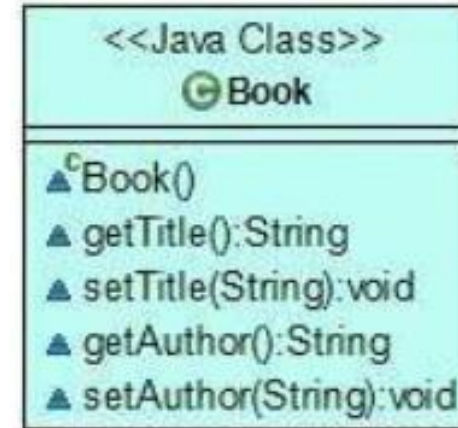
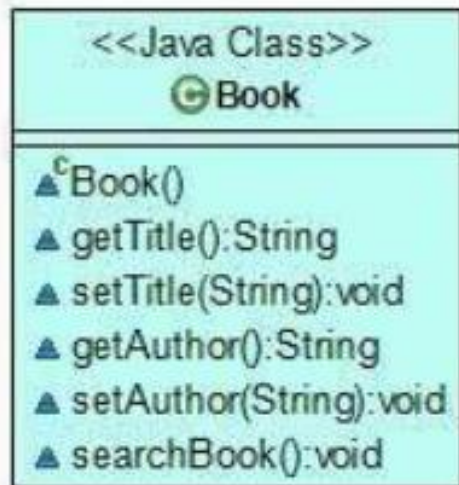
To apply the Single Responsibility Principle, we need to decouple the two responsibilities. In the refactored code, the Book class will only be responsible for getting and setting the data of the Book object.

The Refactored Book Class

```
class Book {  
  
    String title;  
    String author;  
  
    String getTitle() {  
        return title;  
    }  
    void setTitle(String title) {  
        this.title = title;  
    }  
    String getAuthor() {  
        return author;  
    }  
    void setAuthor(String author) {  
        this.author = author;  
    }  
}
```

```
class InventoryView {  
  
    Book book;  
  
    InventoryView(Book book) {  
        this.book = book;  
    }  
  
    void searchBook() {...}  
  
}
```

In UML



A new requirement

- ▶ Now, the store wants to hand out cookbooks at a discount price before Christmas.
- ▶ We already follow the Single Responsibility Principle, so we create two separate classes: CookbookDiscount to hold the details of the discount and DiscountManager to apply the discount to the price.

Adding CookbookDiscount

```
class CookbookDiscount {  
  
    String getCookbookDiscount() {  
  
        String discount = "30% between Dec 1 and 24.";  
  
        return discount;  
    }  
}  
  
class DiscountManager {  
  
    void processCookbookDiscount(CookbookDiscount discount) {...}  
}
```

Another Discount Needed

- ▶ This code works fine until the store management informs us that their cookbook discount sales were so successful that they want to extend it.
- ▶ Now, they want to hand out every biography with a 50% discount.
- ▶ To add the new feature, we create a new BiographyDiscount class:

BiographyDiscount Class

```
class BiographyDiscount {  
  
    String getBiographyDiscount() {  
  
        String discount = "50% on the subject's birthday.";  
  
        return discount;  
  
    }  
}
```

```
class DiscountManager {  
  
    void processCookbookDiscount(CookbookDiscount discount) {...}  
  
    void processBiographyDiscount(BiographyDiscount discount) {...}  
  
}
```

OoPs!!

- ▶ However, as we changed existing functionality, we **violated the Open/Closed Principle**.
- ▶ Although the above code works properly, it might add new vulnerabilities to the application. We don't know how the new addition would interact with other parts of the code that depends on the DiscountManager class. In a real-world application, this would mean that we need to test and deploy our entire app again.
- ▶ But, we can also choose to refactor our code by adding an **extra layer of abstraction** that represents all types of discounts. So, let's create a new interface call.

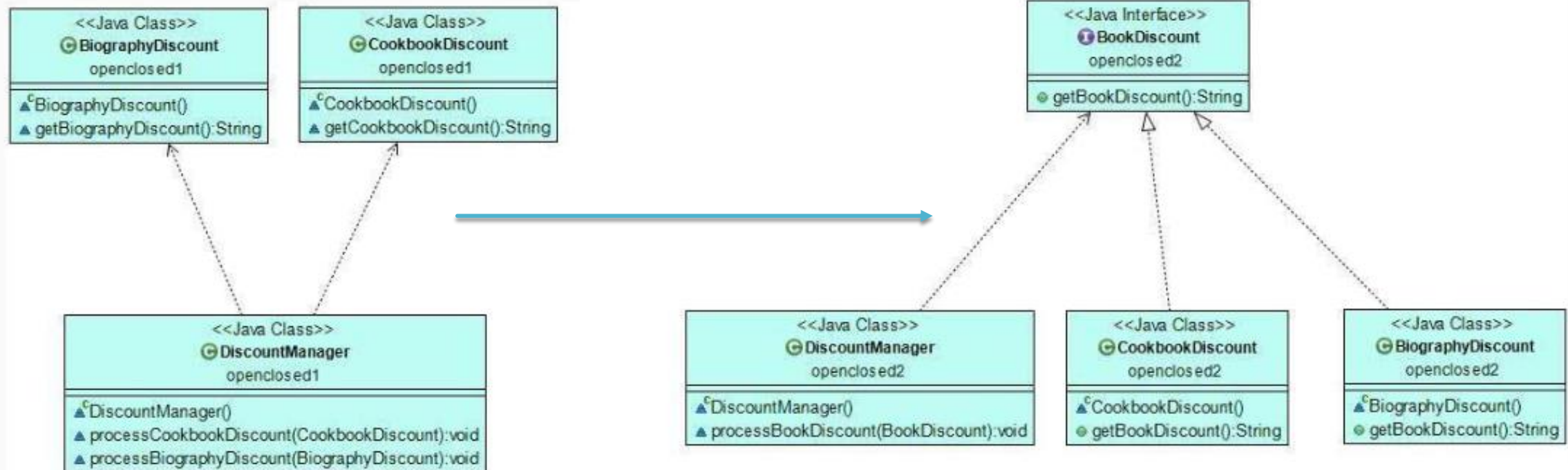
Adding BookDiscount Interface

```
public interface BookDiscount {  
  
    String getBookDiscount();  
  
}
```

```
class DiscountManager {  
  
    void processBookDiscount(BookDiscount discount) {...}  
  
}
```

```
class CookbookDiscount implements BookDiscount {  
  
    @Override  
    public String getBookDiscount() {  
        String discount = "30% between Dec 1 and 24.";  
  
        return discount;  
    }  
  
}
```

In UML



New Requirement ... again

- ▶ Now, the book store asks us to add a new delivery functionality to the application.
- ▶ So, we create a **BookDelivery** class that informs customers about the number of locations where they can collect their order:

```
class BookDelivery {  
    String titles;  
    int userID;  
  
    void getDeliveryLocations() {...}  
}
```

Adding HardcoverDelivery

- ▶ However, the store also sells fancy hardcovers they only want to deliver to their high street shops.
- ▶ So, we create a new HardcoverDelivery subclass that extends BookDelivery and overrides the getDeliveryLocations() method with its own functionality:

```
class HardcoverDelivery extends BookDelivery {  
  
    @Override  
    void getDeliveryLocations() {...}  
  
}
```


Adding AudioBookDelivery

```
class AudiobookDelivery extends BookDelivery {  
  
    @Override  
    void getDeliveryLocations() { /* can't be implemented */ }  
}
```

we realize that audiobooks can't be delivered to physical locations.

We could change some characteristics of the `getDeliveryLocations()` method, however, that would violate the Liskov Substitution Principle.

Fix the inheritance hierarchy

- ▶ Let's introduce an extra layer that better differentiates book delivery types. The `OfflineDelivery` and `OnlineDelivery`.

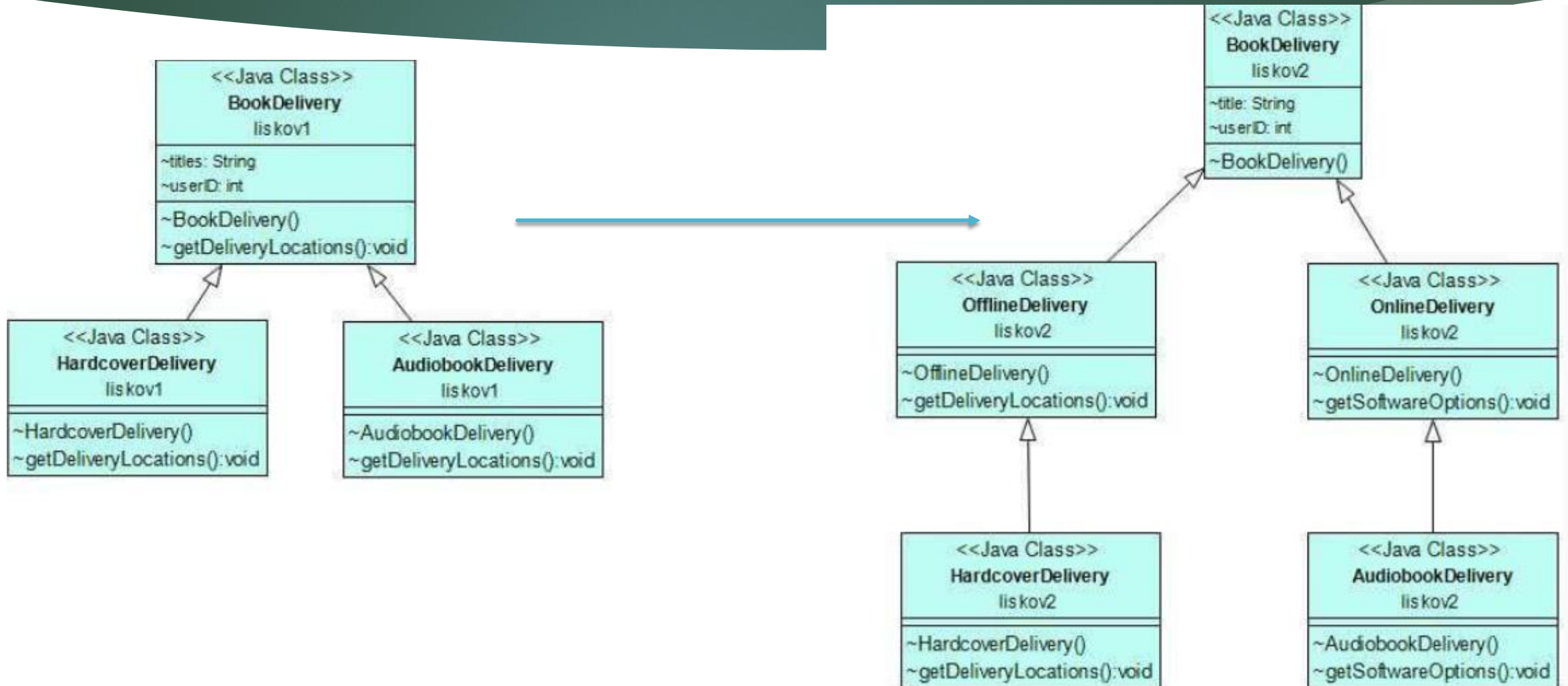
OfflineDelivery vs OnlineDelivery

```
class BookDelivery {  
  
    String title;  
    int userID;  
  
}  
  
class OfflineDelivery extends BookDelivery {  
  
    void getDeliveryLocations() {...}  
  
}  
  
class OnlineDelivery extends BookDelivery {
```

```
class HardcoverDelivery extends OfflineDelivery {  
  
    @Override  
    void getDeliveryLocations() {...}  
  
}  
  
class AudiobookDelivery extends OnlineDelivery {  
  
    @Override
```

After the refactoring, we could use any subclass in place of its superclass without breaking the application.

In UML



Adding extra user actions

- ▶ Let's add some user actions to our online bookstore so that customers can interact with the content before making a purchase. To do so, we create an interface called `BookAction` with three methods: `seeReviews()`, `searchSecondHand()`, and `listenSample()`.

```
public interface BookAction {  
  
    void seeReviews();  
    void searchSecondhand();  
    void listenSample();  
  
}
```

Adding UIs

```
class HardcoverUI implements BookAction {  
  
    @Override  
    public void seeReviews() {...}  
  
    @Override  
    public void searchSecondhand() {...}  
  
    @Override  
    public void listenSample() {...}  
  
}
```

```
class AudiobookUI implements BookAction {  
  
    @Override  
    public void seeReviews() {...}  
  
    @Override  
    public void searchSecondhand() {...}  
  
    @Override  
    public void listenSample() {...}  
  
}
```

Both classes depend on methods they don't use, so we have broken the Interface Segregation Principle.

more specific sub-interfaces: HardcoverAction and AudioAction.

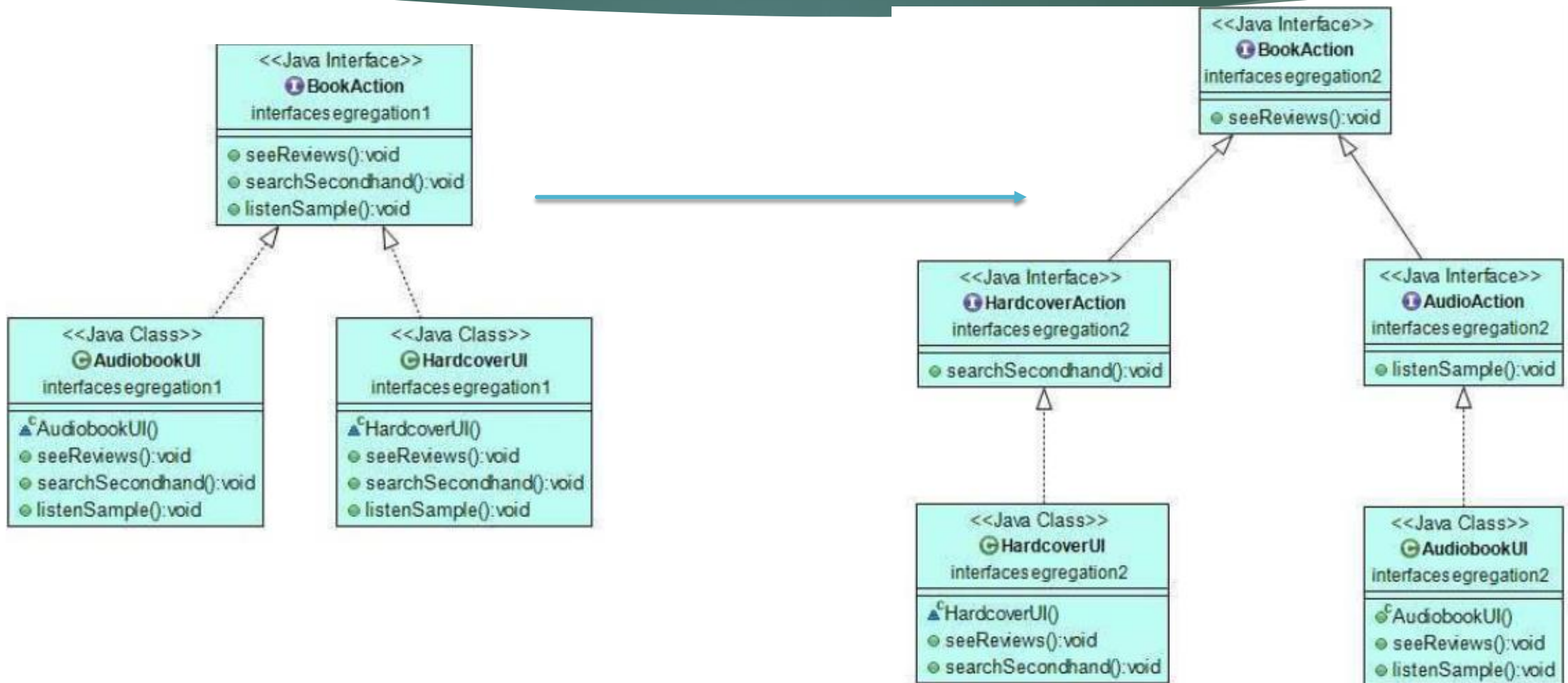
```
public interface BookAction {  
  
    void seeReviews();  
  
}  
  
public interface HardcoverAction extends BookAction {  
  
    void searchSecondhand();  
  
}  
  
public interface AudioAction extends BookAction {  
  
    void
```

```
class HardcoverUI implements HardcoverAction {  
  
    @Override  
    public void seeReviews() {...}  
  
    @Override  
    public void searchSecondhand() {...}  
  
}
```

```
class AudiobookUI implements AudioAction {  
  
    @Override  
    public void seeReviews() {...}
```

segregated interfaces lead to simpler classes that only implement the methods they really need

In UML



A new feature

- ▶ customers can put their favorite books on a shelf.

```
class Book {  
  
    void seeReviews() {...}  
    void readSample() {...}  
  
}  
  
class Shelf {  
  
    Book book;  
  
    void addBook(Book book) {...}  
    void customizeShelf() {...}  
  
}
```

Adding Dvds with Books

- Everything looks fine, but as the high-level Shelf class depends on the low-level Book, the above code violates the **Dependency Inversion Principle**. This becomes clear when the store asks us to enable customers to add DVDs to their shelves, too. To fulfill the demand, we create a new DVD class

```
class DVD {  
  
    void seeReviews() {...}  
    void watchSample() {...}  
  
}
```

Modify the Shelf class needed

- ▶ Now, we should modify the Shelf class so that it can accept DVDs, too.
- ▶ However, this would clearly break the **Open/Closed Principle**.
- ▶ The solution is to create an abstraction layer for the lower-level classes (Book and DVD). We'll do so by introducing the Product interface that both classes will implement.

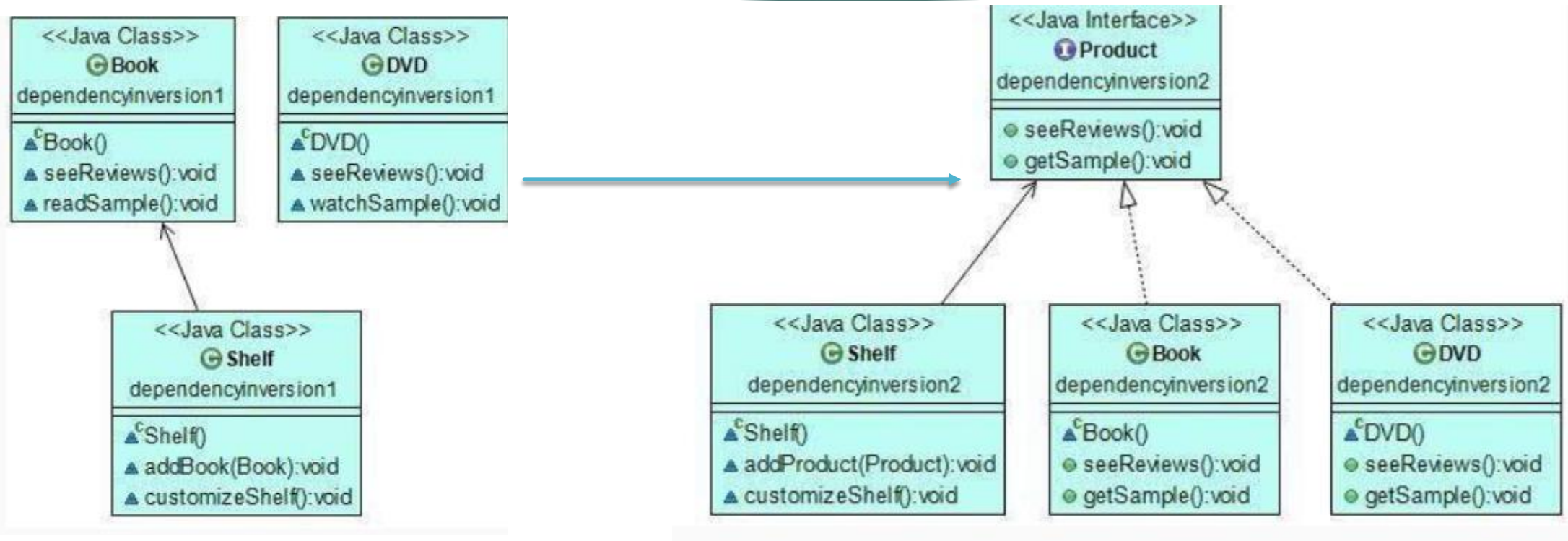
Adding Product Interface

```
public interface Product {  
  
    void seeReviews();  
    void getSample();  
  
}
```

```
class Book implements Product {  
  
    @Override  
    public void seeReviews() {...}  
  
    @Override  
    public void getSample() {...}  
  
}  
  
class DVD implements Product {  
  
    @Override  
    public void seeReviews() {...}  
  
    @Override  
    public void getSample() {...}  
  
}
```

```
class Shelf {  
  
    Product product;  
  
    void addProduct(Product product) {...}  
  
    void customizeShelf() {...}  
  
}
```

In UML



References

- ▶ <https://www.javabrahman.com/programming-principles/liskov-substitution-principal-java-example/>
- ▶ <https://www.oodesign.com/design-principles.html>
- ▶ <https://scotch.io/bar-talk/s-o-l-i-d-the-first-five-principles-of-object-oriented-design>
- ▶ <https://dzone.com/articles/solid-principles-by-examples-liskov-substitution-p>
- ▶ <https://howtodoinjava.com/design-patterns/open-closed-principle/>
- ▶ <https://raygun.com/blog/solid-design-principles/>